

Caso Semestral: "Colegio Bernardo O'Higgins – Plataforma de libro de clases digital"

Integrantes: Vicente Manriquez, Davier Ramos, Dilan Micolta

Asignatura: DESARROLLO FULLSTACK III (DSY1106).

Sección: 2

Fecha: 09/04/2026

Índice

1. Contextualización del caso	3
2. Selección de patrones de arquitectura	3
3. Comunicación entre microservicios y eficiencia	5
4. Arquitectura de contenedores: Seguridad, privacidad y sostenibilidad	6
5. Evaluación general del diseño	7
6. Conclusión	9
7. Bibliografía	10

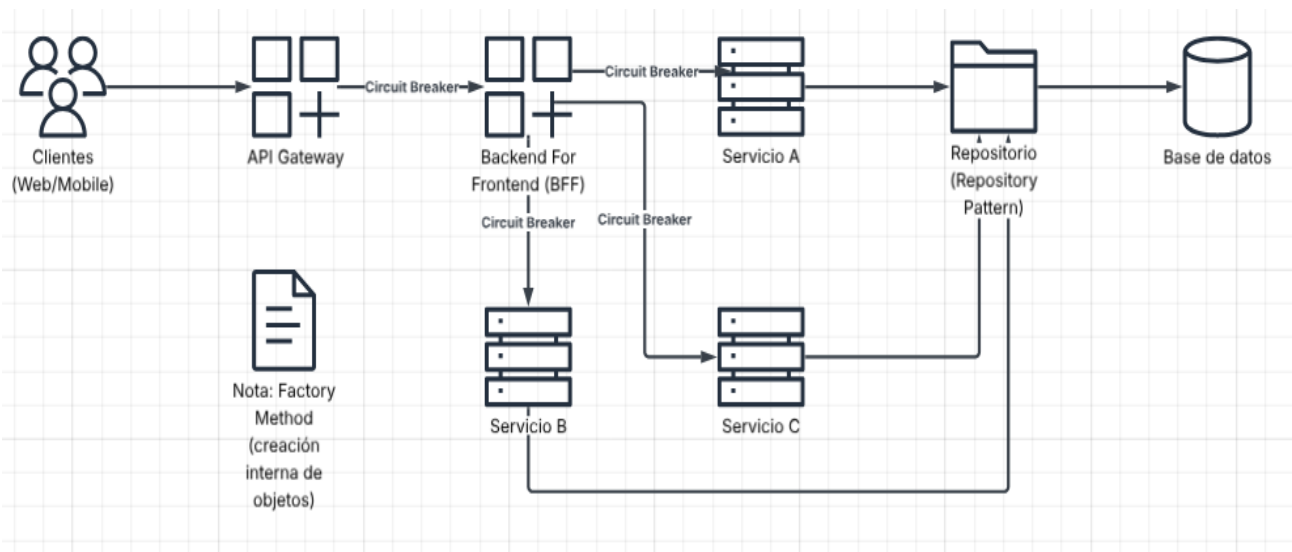
1. Contextualización del caso

El Colegio Bernardo O'Higgins de Coquimbo es una institución de enseñanza básica y media, atendiendo a estudiantes, profesores, apoderados y personal administrativo.

Actualmente, la institución enfrenta diversas dificultades, como la dependencia del libro de clases físico , dificultad para consultar información histórica , falta de herramientas para analizar el desempeño , escasa comunicación y procesos administrativos lentos.

Para mejorar esta situación, el colegio ha decidido desarrollar una plataforma digital que centralice la información académica. Esta solución contempla tres módulos: un sistema de gestión académica , un sistema de registro de asistencia y anotaciones , y un portal de comunicaciones y mensajería. El objetivo final es consolidar una solución integrando todos los módulos desarrollados en un sistema funcional.

2. Selección de patrones de arquitectura



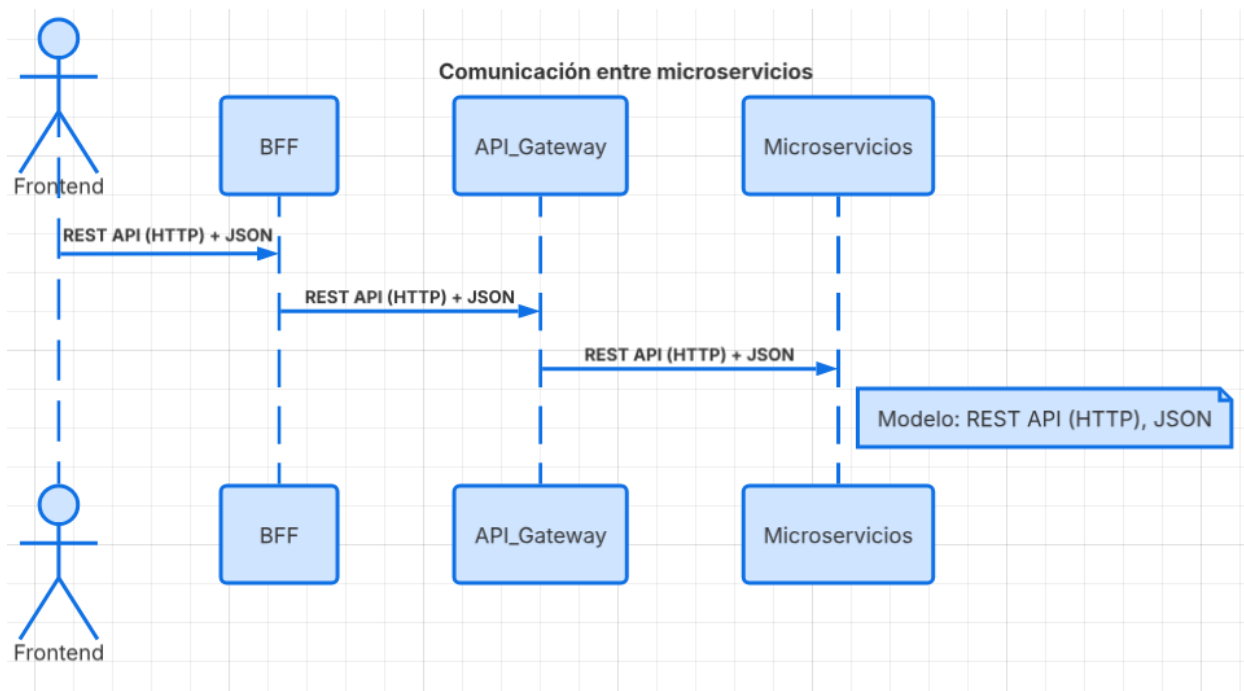
Justificación de la selección: Para cumplir con los requerimientos, se diseñó una arquitectura de microservicios escalable, utilizando Spring Boot sobre un clúster de Kubernetes. Los patrones elegidos permiten construir una solución robusta y acorde a

las necesidades del cliente:

- **API Gateway y Backend For Frontend (BFF):** Se implementa un BFF para gestionar la interacción entre el frontend y los microservicios , mientras que el API Gateway gestiona la comunicación general. Esto simplifica las peticiones desde la interfaz gráfica.
- **Repository Pattern:** Se utiliza para la persistencia de datos , aislando la lógica de negocio del acceso a las bases de datos a través de JPA y entidades.
- **Factory Method:** Implementado para la creación de instancias, lo que facilita la generación dinámica de distintos tipos de reportes o perfiles de usuario.
- **Circuit Breaker:** Fundamental en Kubernetes para manejar fallos en la comunicación entre servicios. Si el servicio de mensajería experimenta caídas, el patrón evita fallos en cascada, permitiendo que el módulo académico siga funcionando.

Estos patrones aseguran que los servicios sean escalables y desacoplados, permitiendo futuras mejoras sin afectar el funcionamiento del sistema.

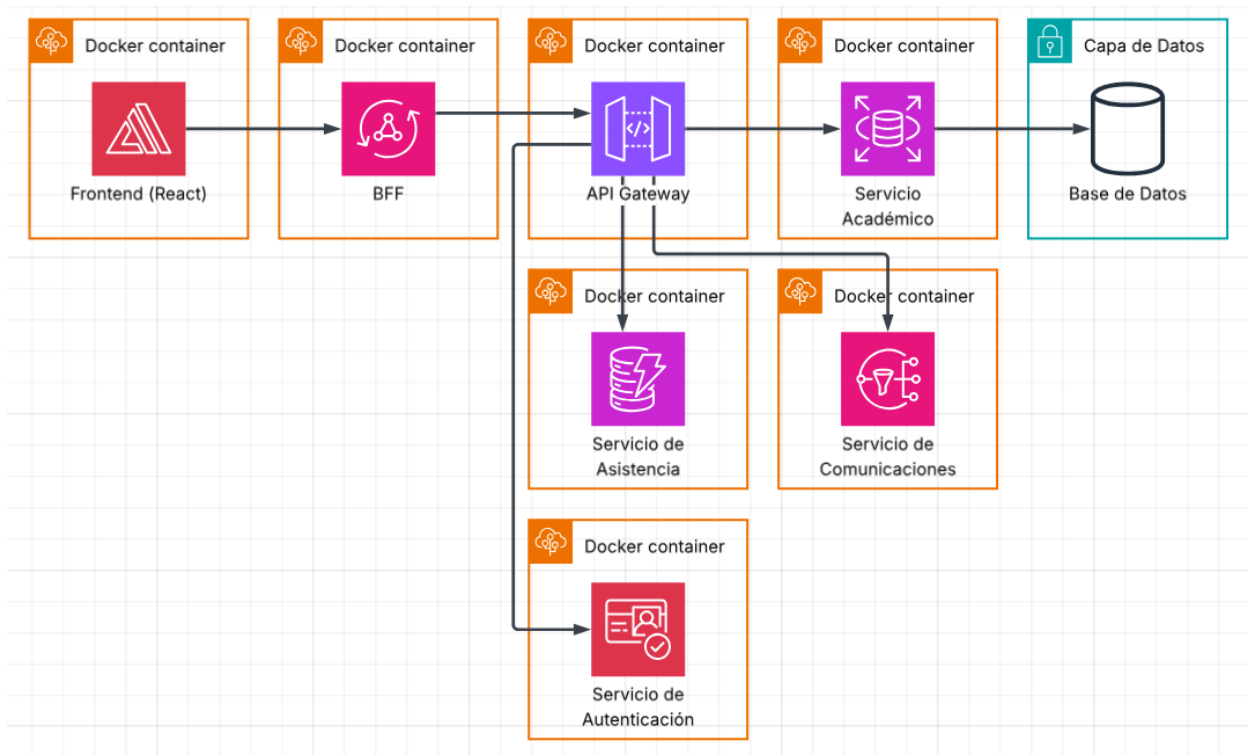
3. Comunicación entre microservicios



Justificación y Eficiencia: La comunicación del sistema inicia en el frontend, el cual interactúa con el backend vía API REST. Las peticiones llegan al API Gateway/BFF, el cual centraliza y enruta el tráfico hacia los al menos dos microservicios independientes.

Esta arquitectura aporta significativamente a la **eficiencia técnica**, ya que Kubernetes gestiona el balanceo de carga y la resolución de nombres de dominio internos (DNS), reduciendo la latencia. Operativamente, el diseño modular y el uso de arquetipos Maven personalizados permiten que equipos de desarrollo distintos trabajen en el módulo de asistencia o en el de comunicaciones en paralelo, agilizando el mantenimiento y despliegue continuo de nuevas versiones.

4. Arquitectura con contenedores: Seguridad, privacidad y sostenibilidad



La implementación de la arquitectura basada en contenedores Docker orquestados por Kubernetes considera los siguientes factores críticos:

- Seguridad:** La contenedorización reduce la superficie de ataque. El sistema define claramente los microservicios clave, asegurando separación de responsabilidades. Solo el API Gateway está expuesto al exterior, mientras que los microservicios internos y las bases de datos permanecen en redes privadas virtuales dentro del clúster.
- Privacidad:** Al conectar microservicios independientes a bases de datos mediante JPA , se garantiza que la información sensible (como el registro de conducta de los estudiantes) esté aislada en su propio contexto de persistencia, impidiendo el cruce no autorizado de información.
- Sostenibilidad:** Kubernetes optimiza dinámicamente el uso de CPU y memoria de los contenedores. A diferencia de las máquinas virtuales tradicionales, esta arquitectura consume solo los recursos estrictamente necesarios según la

demanda en tiempo real, reduciendo la huella energética del centro de datos.

5. Evaluación general del diseño

Debilidades de la arquitectura

1. Alta complejidad inicial

- La arquitectura de microservicios es más compleja que un monolito.
- Requiere:
 - Configuración de múltiples servicios
 - Manejo de redes
 - Orquestación de contenedores

2. Sobrecarga en la comunicación

- Los servicios se comunican por HTTP (REST)
- Esto introduce:
 - Latencia
 - Mayor consumo de red

3. Gestión de datos distribuida

- Cada microservicio puede tener su propia base de datos

Problemas:

- Dificultad en:
 - Transacciones (no hay ACID global)
 - Consistencia de datos

4. Mayor dificultad en debugging

- Un error puede involucrar varios servicios

Problema:

- Es más difícil rastrear fallos que en un sistema monolítico

5. Costos operativos mayores

- Más infraestructura:
 - Contenedores
 - Servidores
 - Monitoreo

2. Riesgos de seguridad

- Más puntos de entrada (APIs)

Riesgos:

- Ataques tipo:
 - API abuse
 - Inyección

3. Problemas de rendimiento

- Muchas llamadas entre servicios

4. Desacoplamiento mal implementado

- Si los servicios no están bien definidos:
Se genera dependencia entre ellos

4. Orquestación con Kubernetes

- Para manejar contenedores

6. API Gateway más robusto

- Rate limiting
- Autenticación centralizada

6. Conclusión

La implementación de esta plataforma de libro de clases digital representa un avance fundamental para la modernización del Colegio Bernardo O'Higgins. Al combinar frameworks modernos en el frontend con un backend robusto basado en patrones arquitectónicos avanzados y orquestación de contenedores, se entrega una solución técnica superior. Este enfoque no solo resuelve la fragmentación de la información actual, sino que proporciona una base tecnológica segura, resiliente y escalable que acompañará el crecimiento de la comunidad educativa a largo plazo.

7. Bibliografía

- *Documento del Caso Semestral: "Colegio Bernardo O'Higgins – Plataforma de libro*

de clases digital".

- VMware. (2024). *Spring Boot Reference Documentation*. Spring.
<https://spring.io/projects/spring-boot>
 - Kubernetes Authors. (2024). *Kubernetes Documentation*. Kubernetes.
<https://kubernetes.io/docs/home/>
 - Docker Inc. (2024). *Docker Documentation*. Docker. <https://docs.docker.com/>
-